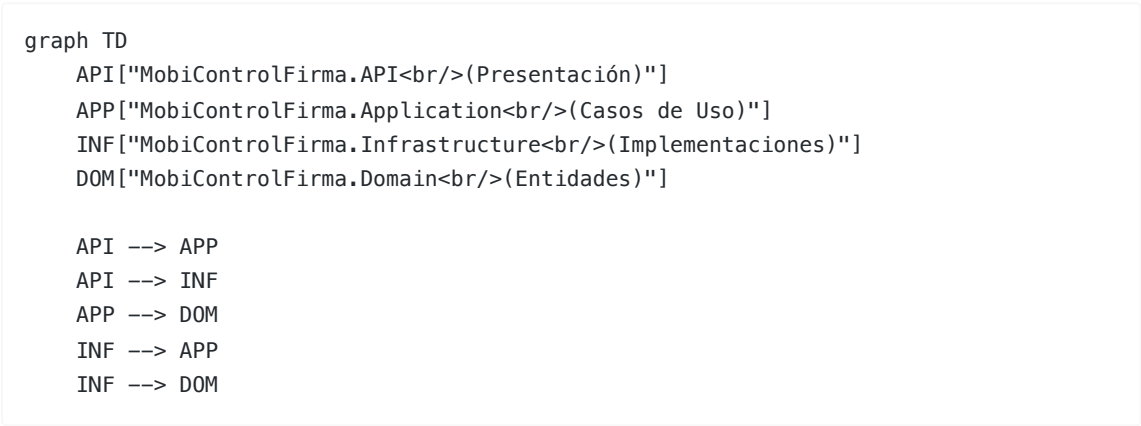


Análisis Completo: firma-digital-api

Documento de referencia para replicar la arquitectura, despliegue Azure e integración con One en otro proyecto.

1. Arquitectura del Proyecto (Clean Architecture)



Capa	Responsabilidad	Depende de
Domain	Entidades, enums, interfaces IDeEmpresa	Nada
Application	Interfaces de servicio, DTOs, excepciones de negocio	Domain
Infrastructure	EF Core, Azure Blob, clientes HTTP, One, correos, PDF	Application, Domain
API	Controllers, middleware, autenticación, Program.cs	Application, Infrastructure

Target Framework

- .NET 10 (net10.0) con C# 14, ImplicitUsings , Nullable habilitado.

2. Program.cs — Pipeline Completo

El orden del middleware es **crítico**. Aquí está la secuencia exacta:

```
// =====
// 1. SERVICIOS (ConfigureServices)
// =====

// Opciones fuertemente tipadas
builder.Services.Configure<AppOptions>(builder.Configuration.GetSection("App"));
builder.Services.Configure<SeguridadOptions>
(builder.Configuration.GetSection("Seguridad"));
builder.Services.Configure<OneOptions>(builder.Configuration.GetSection("One"));
```

```

// Multi-tenant
builder.Services.AddScoped<IContextoEmpresa, ContextoEmpresa>();
builder.Services.AddScoped<EmpresasOne>();
builder.Services.AddScoped<CredencialesOne>();

// Data Protection (para tokens cifrados en URLs)
builder.Services.AddDataProtection().SetApplicationName("MobiControlFirma");
builder.Services.AddSingleton<IEnlacesFirma, EnlacesFirma>();

// Caché en memoria (para tokens de One)
builder.Services.AddMemoryCache();

// HttpClient para autenticación con One
builder.Services.AddHttpClient("one-auth", (sp, client) => {
    var opts = sp.GetRequiredService<IOptions<OneOptions>>().Value;
    client.BaseAddress = new Uri(opts.BaseUrl);
    client.Timeout = TimeSpan.FromSeconds(15);
});

// Autenticación delegada en One
builder.Services.AddAuthentication("One")
    .AddScheme<AuthenticationSchemeOptions, ManejadorAutenticacionOne>("One", null);

// Rate Limiting
builder.Services.AddRateLimiter(PoliticasLimite.Configurar);

// CORS
builder.Services.AddCors(...);

// Infrastructure DI (DB, Blob, HTTP clients, hosted services)
builder.Services.AddInfrastructure(builder.Configuration);

// Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// =====
// 2. MIDDLEWARE PIPELINE (orden estricto)
// =====

// Paso 1: Headers del proxy de Azure
app.UseForwardedHeaders(); // XForwardedFor + XForwardedProto

// Paso 2: Migraciones automáticas + Seed
await db.Database.MigrateAsync();
await ApplicationDbContextSeed.SeedAsync(db);

// Paso 3: Swagger
app.UseSwagger();
app.UseSwaggerUI();

// Paso 4: CORS (temprano para que errores lleven headers CORS)

```

```

app.UseCors("MobiControlFirmaCors");

// Paso 5: Archivos estáticos (SPA React)
app.UseDefaultFiles();
app.UseStaticFiles();

// Paso 6: Auth
app.UseAuthentication();
app.UseAuthorization();

// Paso 7: Resolución de tenant
app.UseMiddleware<ResolucionTenantMiddleware>();

// Paso 8: Manejo global de errores (try/catch middleware)

// Paso 9: Rate limiter
app.UseRateLimiter();

// Paso 10: Controllers + fallback SPA
app.MapControllers().RequireRateLimiting("general");
app.MapFallback(/* sirve index.html para rutas no-API */);

```

3. Autenticación — 4 Mecanismos

```

flowchart TD
    REQ["Petición HTTP entrante"]
    REQ --> CHECK1{"¿Qué credencial trae?"}

    CHECK1 --> AUTH["Authorization: Bearer token"]
    ONE["ManejadorAutenticacionOne<br/>Esquema 'One'"]
    CHECK1 -->|X-Api-Key + X-Api-Secret| CRED["CredencialesOne<br/>Integración externa"]
    CHECK1 -->|X-Api-Key sola| DEVICE["ApiKeyAttribute<br/>Dispositivo o Admin"]
    CHECK1 -->|Token cifrado en URL| LINK["EnlaceFirmaAttribute<br/>Data Protection"]

    ONE -->|GET api/v1/auth/me| ONEAPI["One API"]
    CRED -->|GET api/v1/integration/verify| ONEAPI
    DEVICE -->|SHA256 vs BD| DB["Tabla Empresas"]
    LINK -->|Descifra token| DB2["Tabla SolicitudesFirma"]

```

A. Token de Usuario (Consola SPA) → One

```

// ManejadorAutenticacionOne.cs
// 1. Recibe Bearer token
// 2. Hash SHA-256 del token como clave de caché: "one:me:<hash>"
// 3. Si no está en caché → GET api/v1/auth/me (Bearer <token>)
// 4. Construye ClaimsPrincipal con: NameIdentifier, Email, Name, Roles, Claims

```

```
"tenant"  
// 5. Caché por 1 minuto
```

[!IMPORTANT] Decisión de seguridad clave: La API **NO valida firmas JWT localmente**. Esto evita tener la llave secreta de One en este repositorio. Si se filtrara, un atacante podría forjar tokens para cualquier usuario/empresa.

B. Credenciales de Integración → One

```
// CredencialesOne.cs  
// 1. Headers: X-API-Key + X-API-Secret  
// 2. Caché SHA-256: "one:cred:<sha256(key\nsecret)>" por 2 min  
// 3. GET api/v1/integration/verify → valida active + appSlug == "firma-digital"  
// 4. Si One responde 5xx → HTTP 503 (no 401) para que el cliente reintente
```

C. Llave de Dispositivo → BD Local

```
// ApiKeyAttribute.cs  
// 1. Header X-API-Key (o ?apiKey= en query)  
// 2. Si coincide con ApiKeyAdministrador → FixedTimeEquals → SuperAdmin  
// 3. Si no → SHA256(key) → busca en Empresas.ApiKeyHash
```

D. Enlace de Firma → Data Protection

```
// EnlaceFirmaAttribute.cs + EnlacesFirma.cs  
// 1. Token en URL: /api/v1/firmas/{token}  
// 2. Descifra con DataProtection (propósito: "MobiControlFirma.EnlacesFirma.v1")  
// 3. Obtiene solicitudUid → busca SolicitudFirma → resuelve EmpresaId
```

4. Multi-Tenancy (Multi-Empresa)

Patrón

Base de datos compartida + columna discriminadora **EmpresaId** + Global Query Filters de EF Core.

```
// ApplicationDbContext.cs  
// Todas las entidades con IDeEmpresa reciben:  
builder.Entity<T>().HasQueryFilter(x => SinFiltro || x.EmpresaId == EmpresaFiltro);  
  
// SinFiltro = true cuando:  
// - No hay IContextoEmpresa (migraciones/diseño)  
// - El usuario es PlatformAdmin sin tenant seleccionado
```

Flujo de Resolución del Tenant

```
sequenceDiagram
    participant C as Cliente
    participant MW as ResolucionTenantMiddleware
    participant DB as Base de Datos
    participant EF as EF Core Query Filter

    C->>MW: Request + X-Tenant-Id: <guid>
    MW->>MW: ¿Es dispositivo/integración?<br/>→ EmpresaId ya fijado por
    ApiKeyAttribute
    MW->>MW: ¿Es usuario autenticado?
    MW->>MW: Leer X-Tenant-Id header
    MW->>MW: Validar claim "tenant" del usuario
    MW->>DB: SELECT Id FROM Empresas<br/>WHERE OneTenantId = @guid
    MW->>MW: HttpContext.Items["EmpresaId"] = id
    MW->>EF: ContextoEmpresa.EmpresaId = id
    EF->>EF: WHERE EmpresaId = @id (automático)
```

Auto-Provisión de Empresas

```
// EmpresasOne.cs - SincronizarAsync()
// 1. Consulta One: GET api/v1/me/apps/firma-digital/tenants
// 2. Si empresa existe por OneTenantId → actualiza nombre y slug
// 3. Si empresa fue recreada en One (mismo slug, nuevo GUID) → re-vincula
// 4. Si es nueva → crea fila + genera ApiKey dispositivos + siembra catálogos
```

5. Integración con One — Configuración Dinámica

Opciones (appsettings.json)

```
{
  "One": {
    "BaseUrl": "https://intechsys-one-api-....azurewebsites.net"
  }
}
```

```
// OneOptions.cs
public class OneOptions
{
    public const string SectionName = "One";
    public string BaseUrl { get; set; } = string.Empty;
    public string AppSlug { get; set; } = "firma-digital"; // Tu app slug aquí
    public bool EstaConfigurado => !string.IsNullOrEmpty(BaseUrl);
}

// Constantes del protocolo
public static class One
{

```

```

    public const string ClaimTenant = "tenant";           // Claim: "{tenantId}:"
    {rol}"
    public const string RolPlataforma = "PlatformAdmin";
    public const string CabeceraTenant = "X-Tenant-Id";
}

```

ProveedorConfiguracionOne — Configuración por Empresa desde One

```

// Flujo:
// 1. Caché: "one:config:{empresaId}" → 5 minutos
// 2. Lee OneApiKey y OneApiSecret de tabla Empresas
// 3. GET api/v1/integration/config (X-API-Key, X-API-Secret)
// 4. Mapea settings dict → ConfiguracionEmpresa tipada
// 5. Incluye: URLs de servicios, credenciales MobiControl, Infobip, webhooks, etc.

```

*[!TIP] La configuración de servicios externos (MobiControl, Infobip, webhooks) **no se almacena en appsettings ni en BD local**, sino que se consulta dinámicamente a One. Esto permite cambiar configuraciones sin redespiegue.*

6. Despliegue en Azure

Servicios Azure Utilizados

Servicio	Recurso	Uso
App Service (Linux)	fiirmadigital-api	Hosting del API .NET 10 + SPA estática
Azure SQL Database	FirmaDigitalDB en firmadigitalserver	Persistencia EF Core
Azure Blob Storage	Contenedores firmas y documentos-pdf	Imágenes de firma y PDFs
Azure Static Web Apps	Frontend React (consola admin)	SPA administrativa
One API (App Service)	intechsys-one-api-...	IAM centralizado

CI/CD Pipeline (GitHub Actions)

```

flowchart LR
    A["Push a main<br/>o manual"] --> B["Job: build"]
    B --> C["Job: migrate"]
    B --> D["Job: deploy"]
    C --> D

    subgraph build ["build (ubuntu-latest)"]
        B1["Setup .NET 10"] --> B2["Limpiar bin/obj"]
    end

```

```

    B2 --> B3["dotnet build --config Release"]
    B3 --> B4["dotnet publish<br/>SOL0 MobiControlFirma.API.csproj"]
    B4 --> B5["Upload artifact"]
end

subgraph migrate ["migrate"]
    C1["Install dotnet-ef 10.*"] --> C2["dotnet restore"]
    C2 --> C3["dotnet ef database update<br/>ASPNETCORE_ENVIRONMENT=Production"]
end

subgraph deploy ["deploy"]
    D1["Download artifact"] --> D2["Crear app.zip"]
    D2 --> D3["azure/webapps-deploy@v3<br/>type: zip, clean: true, restart:
true"]
end

```

[!WARNING] **Decisiones críticas del pipeline:**

- Se publica **solo el proyecto API** (no la solución completa) para evitar conflictos con `runtimeconfig.json` de Infrastructure.
- Se usa `type: zip` (OneDeploy) en vez de ZipDeploy estándar para evitar file-locks en Linux.
- `clean: true` purga `wwwroot` previo. `restart: true` fuerza reinicio limpio.
- Las **migraciones corren ANTES del deploy** para que si fallan, el código anterior siga funcionando.

Archivo del Workflow

```

# .github/workflows/main_firmadigitalapi.yml
name: Build and deploy to Azure Web App

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.x'
      - run: find . -type d \( -name obj -o -name bin \) -prune -exec rm -rf {} +
      - run: dotnet build --configuration Release
      - run: dotnet publish MobiControlFirma.API/MobiControlFirma.API.csproj -c
Release -o ${env.DOTNET_ROOT}/myapp
      - uses: actions/upload-artifact@v4
        with:
          name: .net-app
          path: ${env.DOTNET_ROOT}/myapp

```

```

migrate:
  runs-on: ubuntu-latest
  needs: build
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
      with:
        dotnet-version: '10.x'
    - run: dotnet tool install --global dotnet-ef --version 10.*
    - run: find . -type d \( -name obj -o -name bin \) -prune -exec rm -rf {} +
    - run: dotnet restore MobiControlFirma.sln
    - run: ASPNETCORE_ENVIRONMENT=Production dotnet ef database update
      --project MobiControlFirma.Infrastructure
      --startup-project MobiControlFirma.API

deploy:
  runs-on: ubuntu-latest
  needs: [build, migrate]
  steps:
    - uses: actions/download-artifact@v4
      with:
        name: .net-app
    - run: zip -r app.zip .
    - uses: azure/webapps-deploy@v3
      with:
        app-name: 'fiirmadigital-api'
        slot-name: 'Production'
        publish-profile: '${{ secrets.AZUREAPPSERVICE_PUBLISHPROFILE }}'
        package: app.zip
        type: zip
        clean: true
        restart: true

```

7. Infraestructura — Inyección de Dependencias

```

// Infrastructure/DependencyInjection.cs
public static IServiceCollection AddInfrastructure(
    this IServiceCollection services, IConfiguration config)
{
    var cadena = config.GetConnectionString("Default");

    // Base de datos SQL Server
    services.AddDbContext<ApplicationDbContext>(options =>
        options.UseSqlServer(cadena, sql =>

sql.MigrationsAssembly(typeof(ApplicationDbContext).Assembly.FullName));
    services.AddScoped<IApplicationDbContext>(sp =>
        sp.GetRequiredService<ApplicationDbContext>());

```

```

// Almacenamiento: Azure Blob o Local (toggle automático)
var blobConn = config["Almacenamiento:AzureBlobConnectionString"];
if (!string.IsNullOrEmpty(blobConn))
    services.AddSingleton<IAlmacenamientoArchivos>(new
AlmacenamientoAzureBlob(blobConn, ...));
else
    services.AddSingleton<IAlmacenamientoArchivos>(new
AlmacenamientoLocal(...));

// PDF con QuestPDF
QuestPDF.Settings.License = LicenseType.Community;
services.AddSingleton<IGeneradorActaPdf, GeneradorActaPdf>();

// HTTP Clients tipados
services.AddHttpClient<IClienteMobiControl, ClienteMobiControl>(...) // 2 min
timeout
services.AddHttpClient<IProveedorConfiguracion, ProveedorConfiguracionOne>(...)
// 20s
services.AddHttpClient<IEnviadorCorreo, EnviadorCorreoInfobip>(...) // 60s
services.AddHttpClient("callbacks", c => c.Timeout = TimeSpan.FromSeconds(30));

// Background Services
services.AddHostedService<ServicioEnvioCorreos>();
services.AddHostedService<ServicioCallbacks>();

// Servicios de aplicación
services.AddScoped<IServicioEntregas, ServicioEntregas>();
services.AddScoped<IServicioSolicitudes, ServicioSolicitudes>();
services.AddScoped<IServicioGeolocalizacion, ServicioGeolocalizacion>();
}

```

8. Rate Limiting

```

// 3 políticas particionadas por IP remota (FixedWindow):
//
// | Política | Límite / minuto | Uso |
// |-----|-----|-----|
// | "firmas" | 20 | POST firma/entrega (alto costo I/O) |
// | "general" | 300 | Todos los controllers (global) |
// | "cuenta" | 40 | Login / validación de sesión |
// |-----|-----|-----|
// Respuesta 429: { "message": "Demasiadas peticiones..." } + Retry-After: 60

```

9. CORS

```

// Orígenes permitidos (de App:CorsOrigins):
// - https://gentle-river-0ff09870f.3.azurestaticapps.net (frontend Azure)
// - http://localhost:5173 (desarrollo local)

```

```
// - "null" (formulario móvil abierto desde file:///...)
// AllowAnyHeader(), AllowAnyMethod()
```

10. Callbacks / Webhooks

```
sequenceDiagram
    participant BG as ServicioCallbacks<br/>(Background)
    participant DB as Base de Datos
    participant EXT as Sistema Externo
    participant ONE as One API

    loop Cada 20 segundos
        BG->>DB: Buscar solicitudes<br/>callback PENDIENTE o ERROR
        BG->>ONE: GET config (CALLBACK_URL, CALLBACK_SECRET)
        BG->>EXT: POST callback_url<br/>Headers: X-Firma-Evento, X-Firma-Signature<br/>Body: empresa + acta + PDF base64
        alt Éxito
            BG->>DB: EstadoCallback = ENVIADO
        else Error
            BG->>DB: Incrementar intentos<br/>Backoff: 1m, 5m, 15m, 1h, 6h
        end
    end
end
```

Firma HMAC-SHA256: `sha256=HMAC(secret, "{timestamp}.{payload_json}")`

11. Estructura de appsettings.json

```
{
  "ConnectionStrings": {
    "Default": "Server=...;Initial Catalog=...;User Id=...;Password=..."
  },
  "One": {
    "BaseUrl": "https://intechsys-one-api-....azurewebsites.net"
  },
  "App": {
    "CorsOrigins": ["https://...", "http://localhost:5173"],
    "UrlFront": "https://...",
    "UrlApi": "https://..."
  },
  "Seguridad": {
    "ApiKeyAdministrador": "..."
  },
  "Almacenamiento": {
    "AzureBlobConnectionString": "",
    "RutaLocal": "/home/data/almacenamiento",
    "ContenedorFirmas": "firmas",
    "ContenedorDocumentos": "documentos-pdf"
  }
}
```

```
}  
}
```

[!CAUTION] `appsettings.Development.json` está configurado con `CopyToPublishDirectory="Never"` en el `.csproj` para que las credenciales de desarrollo **nunca se publiquen** a Azure.

12. Endpoints del API (Referencia Rápida)

Método	Ruta	Auth	Propósito
GET	/api/v1/salud	Público	Health check
GET	/api/v1/sesion	Bearer (One)	Info del usuario + empresas
POST	/api/v1/solicitudes	ApiKey+Secret (One)	Crear solicitud de firma
GET	/api/v1/solicitudes/{id}	ApiKey+Secret	Estado de solicitud
GET	/api/v1/firmas/{token}	Token cifrado	Formulario de firma
POST	/api/v1/firmas/{token}	Token cifrado	Registrar firma
POST	/api/v1/entregas	ApiKey dispositivo	Registrar acta desde móvil
GET	/api/v1/entregas	Bearer (One)	Listar actas (consola)
GET	/api/v1/vinculos	PlatformAdmin	Gestión de empresas
GET	/api/v1/catalogos/*	ApiKey dispositivo	Distritos, canales, estados
GET	/api/v1/geolocalizacion/*	Bearer (One)	GPS de la flota

13. Checklist para Replicar en Otro Proyecto

Estructura de la Solución

- ☐ Crear 4 proyectos: `*.Domain`, `*.Application`, `*.Infrastructure`, `*.API`
- ☐ Domain sin dependencias; Application depende de Domain; Infrastructure de Application
- ☐ API referencia Infrastructure y Application

Integración con One

- ☐ Crear `OneOptions.cs` con `BaseUrl` y `AppSlug` (cambiar el slug a tu app)
- ☐ Implementar `ManejadorAutenticacionOne` (auth delegada, caché SHA-256 por 1 min)
- ☐ Implementar `CredencialesOne` (validación de integraciones, caché 2 min)
- ☐ Implementar `EmpresasOne` (sincronización dinámica de tenants)
- ☐ Implementar `ProveedorConfiguracionOne` (config dinámica por empresa, caché 5 min)
- ☐ Crear `ResolucionTenantMiddleware` para resolver tenant por request

Multi-Tenancy

- ☐ Interfaz `IDeEmpresa` con propiedad `EmpresaId`

- ☐ `IContextoEmpresa` (Scoped) con `EmpresaId` y `EsSuperAdministrador`
- ☐ Global Query Filters en `DbContext` para todas las entidades con `IDeEmpresa`
- ☐ Tabla `Empresas` con `OneTenantId` , `OneSlug` , `OneApiKey` , `OneApiSecret`

Azure

- ☐ GitHub Actions workflow con 3 jobs: build → migrate → deploy
- ☐ Publicar **solo el proyecto API** (no la solución)
- ☐ Usar `type: zip` + `clean: true` + `restart: true` en `azure/webapps-deploy`
- ☐ Migrar BD **antes** de deploy
- ☐ Configurar `ForwardedHeaders` para IPs reales detrás del proxy
- ☐ Azure Blob Storage con fallback a almacenamiento local
- ☐ `appsettings.Development.json` con `CopyToPublishDirectory="Never"`

Seguridad

- ☐ Rate limiting por IP (firmas: 20/min, general: 300/min, cuenta: 40/min)
- ☐ CORS con origen `"null"` si hay formularios móviles locales
- ☐ Comparación de API keys con `CryptographicOperations.FixedTimeEquals`
- ☐ Webhooks con firma HMAC-SHA256 + reintentos exponenciales